

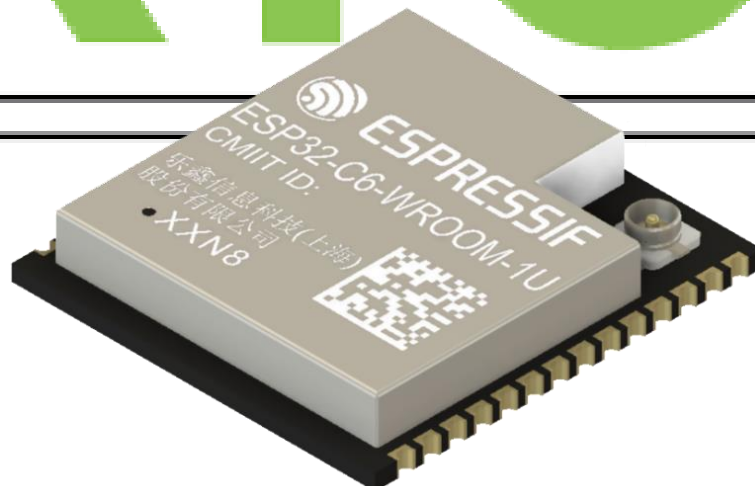
30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 25)

“Inter-Task

Communication: Putting It
All Together”

freeRTOS



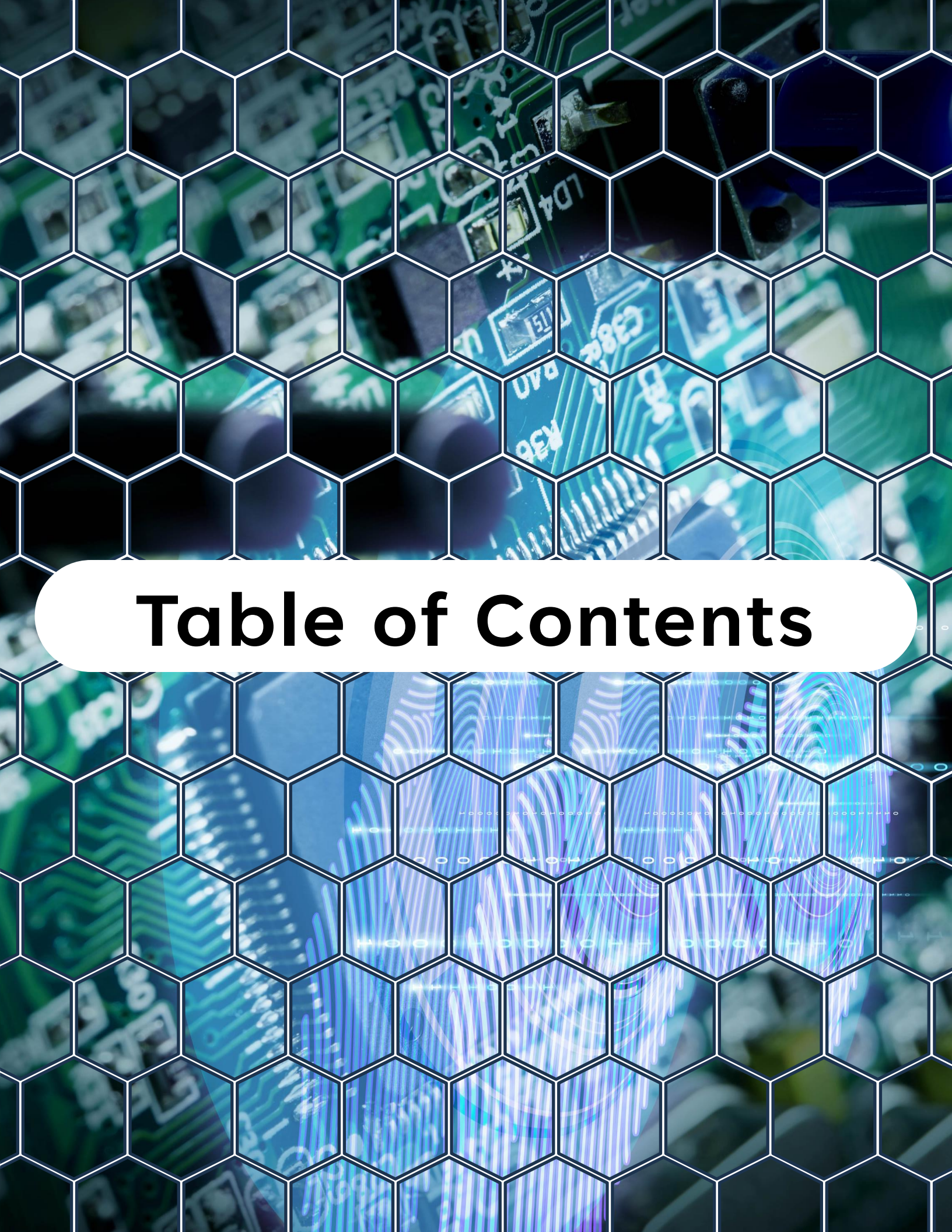
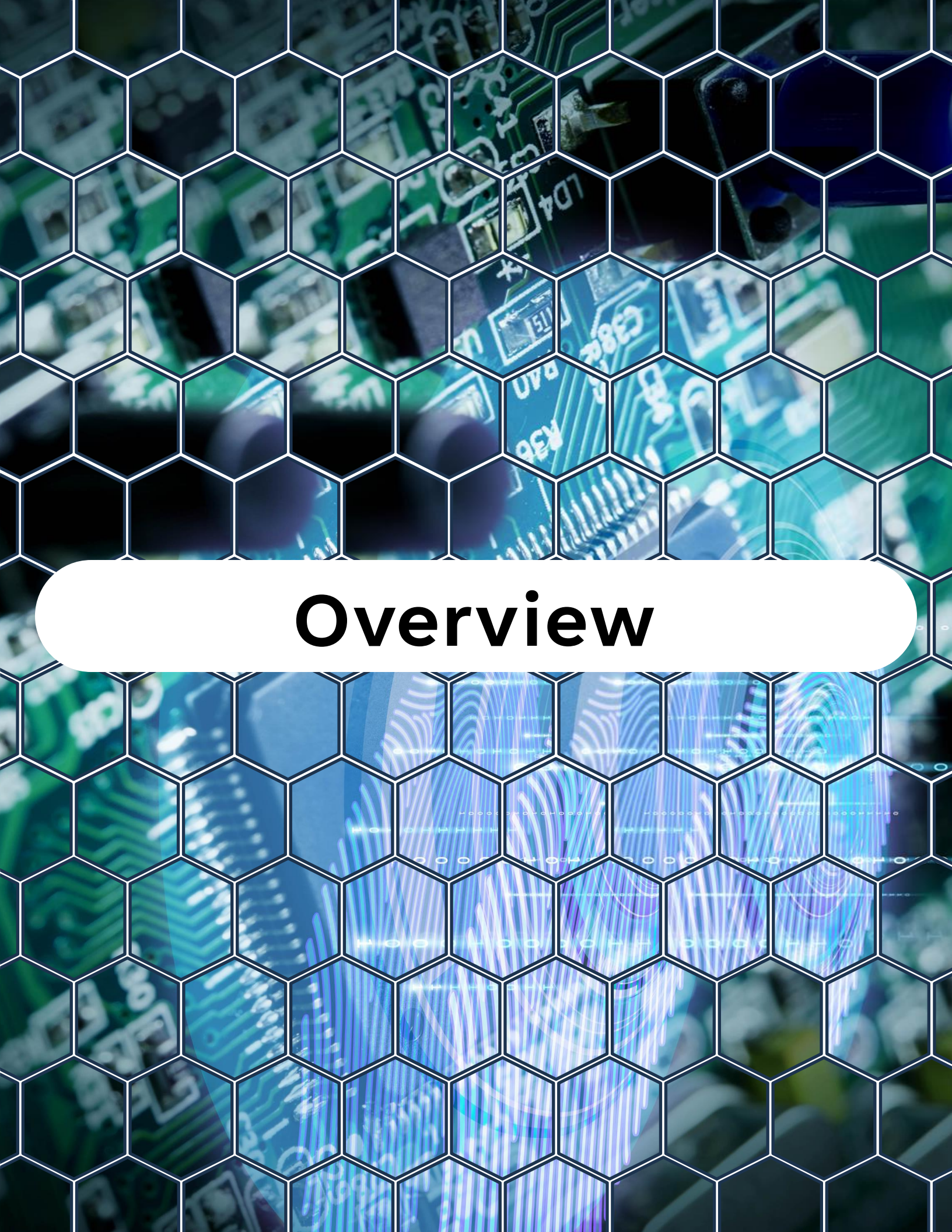


Table of Contents

Table of Contents

1. Overview
2. Review of Communication Mechanisms
3. System Design Example
4. Example: Inter-Task Communication Project
5. Best Practices
6. Summary
7. Challenge for Today



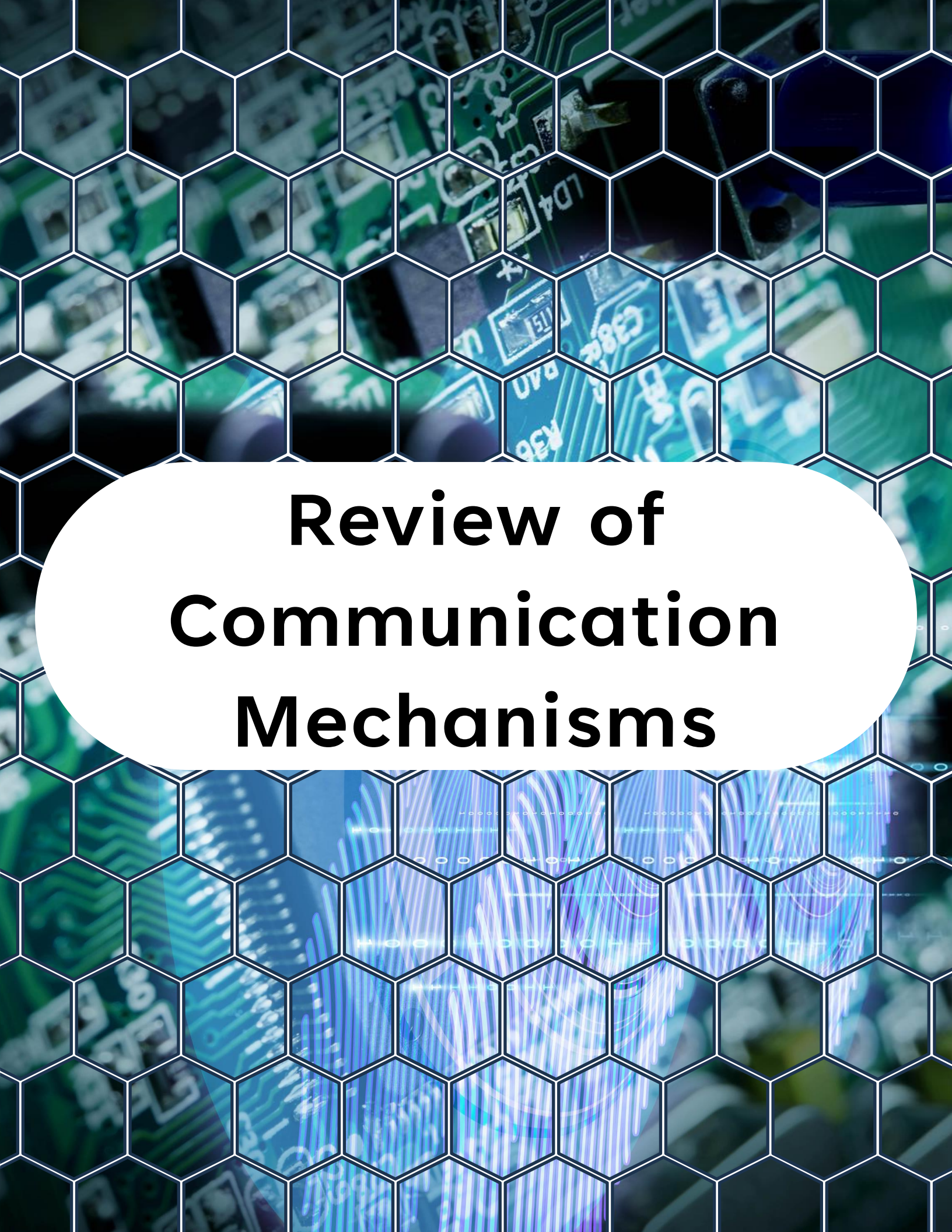
Overview

Overview

On **Day 25**, you'll learn:

- How different FreeRTOS communication primitives (queues, semaphores, event groups, task notifications) can work together
- When to choose each method in real-world designs
- How to combine them in a multi-task ESP32 project
- A practical example: sensor reading + data processing + UART logger
- Best practices for structuring FreeRTOS systems with multiple communication paths

By the end of this lesson, you'll see how to integrate all the tools you've learned so far into a cohesive application.

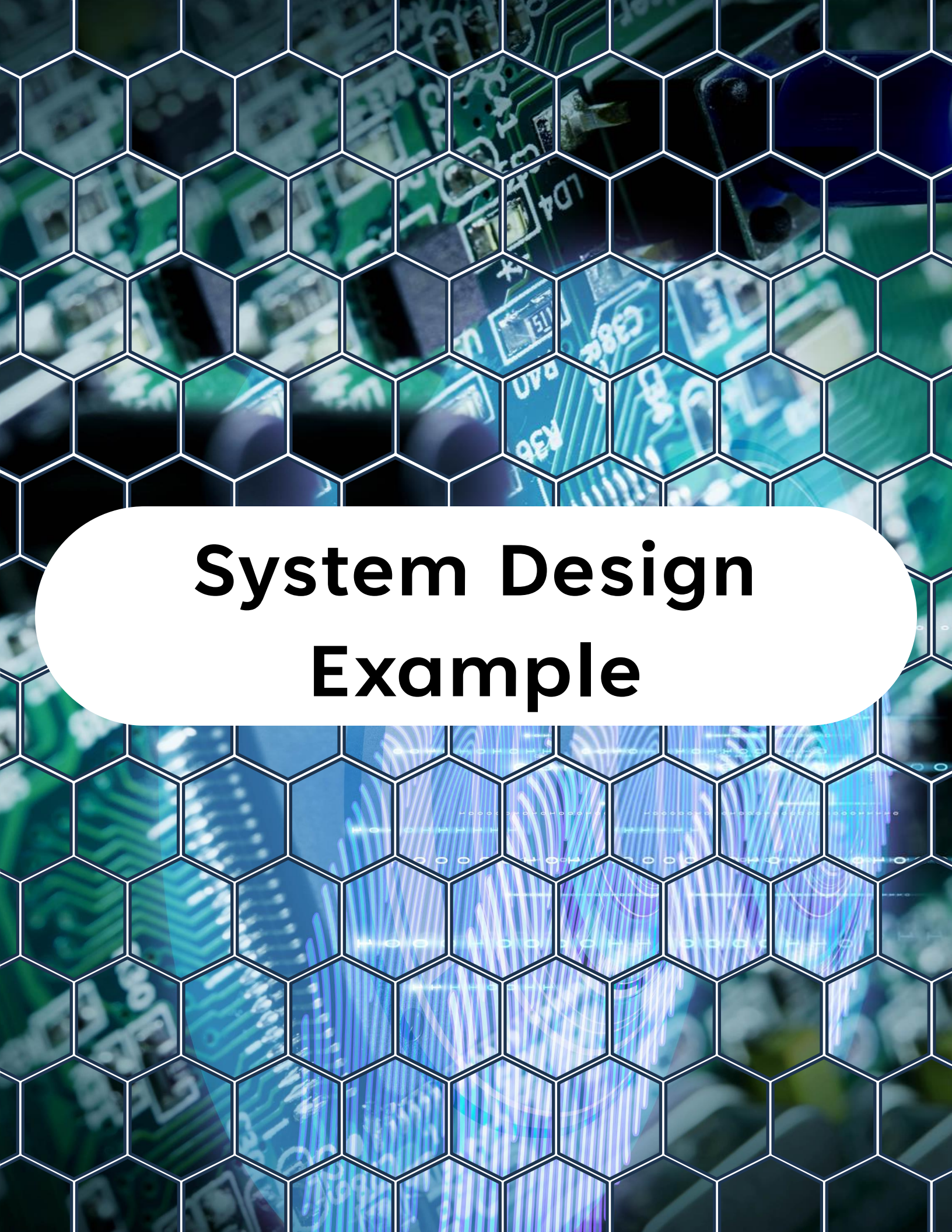


Review of Communication Mechanisms

Review of Communication Mechanisms

Primitive	Best Use Case
Queues	Transfer data (messages, structs) between tasks or ISRs.
Binary Semaphore	Signal an event (ISR-to-task or task-to-task).
Counting Semaphore	Manage access to multiple resources or count events.
Mutex	Protect shared resources (UART, I ² C, file system).
Event Groups	Synchronize multiple tasks using bit flags.
Task Notifications	Lightweight, fast, one-to-one signaling.

 Real-world systems often mix these mechanisms depending on requirements.

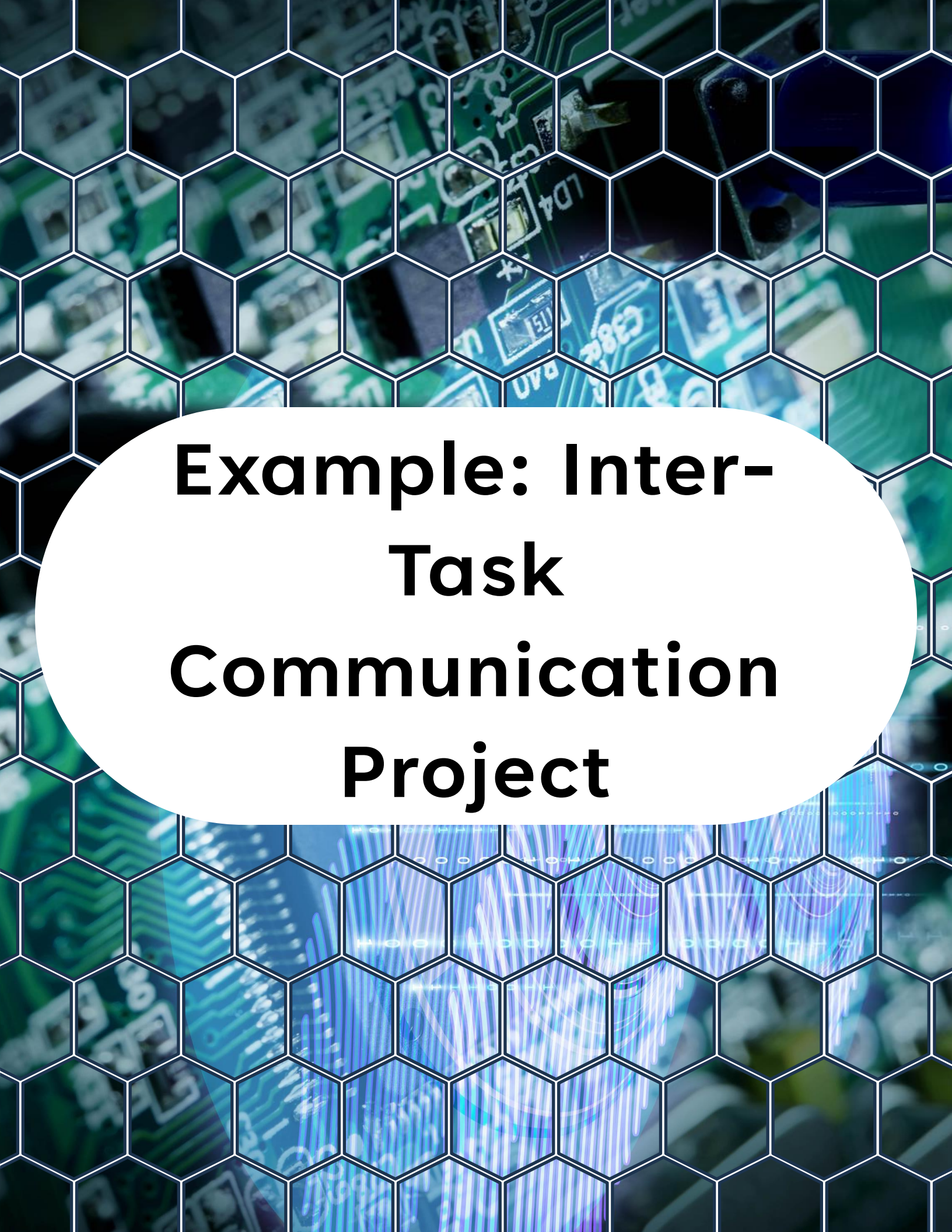


System Design Example

System Design Example

Let's build a **multi-task system**:

- **Sensor Task** → Reads a fake sensor (temperature value) every second.
 - Sends the reading via a **queue**.
- **Processor Task** → Waits on the queue, calculates an average.
 - Uses a **counting semaphore** to track available samples.
- **Logger Task** → Uses a **mutex-protected UART** logger to print results.
- **Event Group** → Ensures both sensor and processor tasks have run at least once before logging.



Example: Inter- Task Communication Project

Example: Inter-Task Communication Project

Code Example



```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "freertos/FreeRTOS.h"
4  #include "freertos/task.h"
5  #include "freertos/queue.h"
6  #include "freertos/semphr.h"
7  #include "freertos/event_groups.h"
8  #include "driver/uart.h"
9  #include "esp_log.h"
10
11 #define TAG "DAY25"
12
13 // Event group bits
14 #define BIT_SENSOR_READY    (1 << 0)
15 #define BIT_PROCESSOR_READY (1 << 1)
16
17 // Handles
18 QueueHandle_t sensor_queue;
19 SemaphoreHandle_t sample_sem;
20 SemaphoreHandle_t uart_mutex;
21 EventGroupHandle_t system_event_group;
22
23 // Sensor task: generate a fake temperature reading
24 void sensor_task(void *pvParameters) {
25     int temp = 25;
26     while (1) {
27         temp = 20 + rand() % 10; // fake data
28         xQueueSend(sensor_queue, &temp, portMAX_DELAY);
29         xSemaphoreGive(sample_sem); // signal new sample
30         xEventGroupSetBits(system_event_group, BIT_SENSOR_READY);
31         vTaskDelay(pdMS_TO_TICKS(1000));
32     }
33 }
34
35 // Processor task: consume samples and compute average
```

Example: Inter-Task Communication Project

```
35 // Processor task: consume samples and compute average
36 void processor_task(void *pvParameters) {
37     int value;
38     int sum = 0, count = 0;
39
40     while (1) {
41         if (xSemaphoreTake(sample_sem, portMAX_DELAY)) {
42             if (xQueueReceive(sensor_queue, &value, 0)) {
43                 sum += value;
44                 count++;
45                 if (count == 5) {
46                     int avg = sum / count;
47                     sum = 0;
48                     count = 0;
49
50                     // Store average in event group as "ready" signal
51                     ESP_LOGI(TAG, "Computed avg temp: %d", avg);
52                     xEventGroupSetBits(system_event_group,
53                                         BIT_PROCESSOR_READY);
54                 }
55             }
56         }
57     }
58 }
59
60 // Logger task: logs only after both sensor and processor are active
61 void logger_task(void *pvParameters) {
62     while (1) {
63         EventBits_t bits = xEventGroupWaitBits(
64             system_event_group,
65             BIT_SENSOR_READY | BIT_PROCESSOR_READY,
66             pdFALSE, // don't clear bits
67             pdTRUE,  // wait for ALL
68             portMAX_DELAY
69         );
70
71         if ((bits & (BIT_SENSOR_READY | BIT_PROCESSOR_READY)) ==
72             (BIT_SENSOR_READY | BIT_PROCESSOR_READY)) {
73             // Both sensor and processor are ready, log the data
74             // ...
75         }
76     }
77 }
```


Example: Inter-Task Communication Project

```
60 // Logger task: logs only after both sensor and processor are active
61 void logger_task(void *pvParameters) {
62     while (1) {
63         EventBits_t bits = xEventGroupWaitBits(
64             system_event_group,
65             BIT_SENSOR_READY | BIT_PROCESSOR_READY,
66             pdFALSE, // don't clear bits
67             pdTRUE,  // wait for ALL
68             portMAX_DELAY
69         );
70
71         if ((bits & (BIT_SENSOR_READY | BIT_PROCESSOR_READY)) ==
72             (BIT_SENSOR_READY | BIT_PROCESSOR_READY)) {
73             if (xSemaphoreTake(uart_mutex, portMAX_DELAY)) {
74                 printf("Logger: Both Sensor & Processor are active\n");
75                 xSemaphoreGive(uart_mutex);
76             }
77             vTaskDelay(pdMS_TO_TICKS(2000));
78         }
79     }
80 }
81
82 // Main application entry point
83 void app_main() {
84     // Create a queue for sensor data
85     sensor_queue = xQueueCreate(10, sizeof(int));
86
87     // Counting semaphore for samples
88     sample_sem = xSemaphoreCreateCounting(10, 0);
89
90     // Mutex for UART access
91     uart_mutex = xSemaphoreCreateMutex();
92
93     // Event group for system state
94     system_event_group = xEventGroupCreate();
95
96     // Create tasks
97     xTaskCreate(sensor_task, "SensorTask", 2048, NULL, 5, NULL);
```

Example: Inter-Task Communication Project

```
81
82 // Main application entry point
83 void app_main() {
84     // Create a queue for sensor data
85     sensor_queue = xQueueCreate(10, sizeof(int));
86
87     // Counting semaphore for samples
88     sample_sem = xSemaphoreCreateCounting(10, 0);
89
90     // Mutex for UART access
91     uart_mutex = xSemaphoreCreateMutex();
92
93     // Event group for system state
94     system_event_group = xEventGroupCreate();
95
96     // Create tasks
97     xTaskCreate(sensor_task, "SensorTask", 2048, NULL, 5, NULL);
98     xTaskCreate(processor_task, "ProcessorTask", 2048, NULL, 6, NULL);
99     xTaskCreate(logger_task, "LoggerTask", 2048, NULL, 4, NULL);
100
101     ESP_LOGI(TAG, "Day 25 system initialized");
102 }
```


Example: Inter-Task Communication Project

Expected Behavior

- **Sensor Task** produces values every second and signals availability.
- **Processor Task** consumes samples, computes an average every 5 readings.
- **Logger Task** waits until both sensor and processor are ready, then prints system status safely (using UART mutex).

Serial output:

I (1000) DAY25: Computed avg temp: 24

Logger: Both Sensor & Processor are active

I (6000) DAY25: Computed avg temp: 22

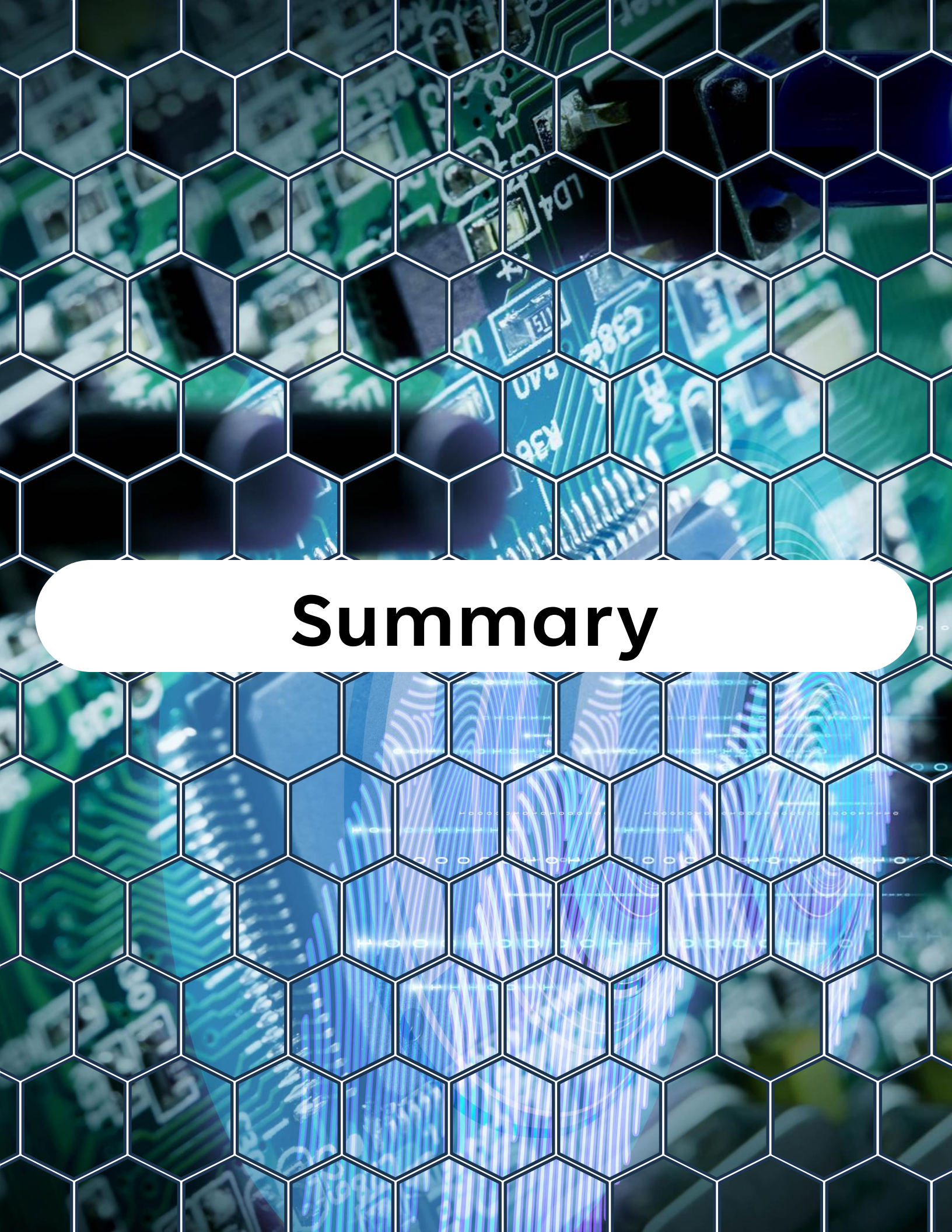
Logger: Both Sensor & Processor are active



Best Practices

Best Practices

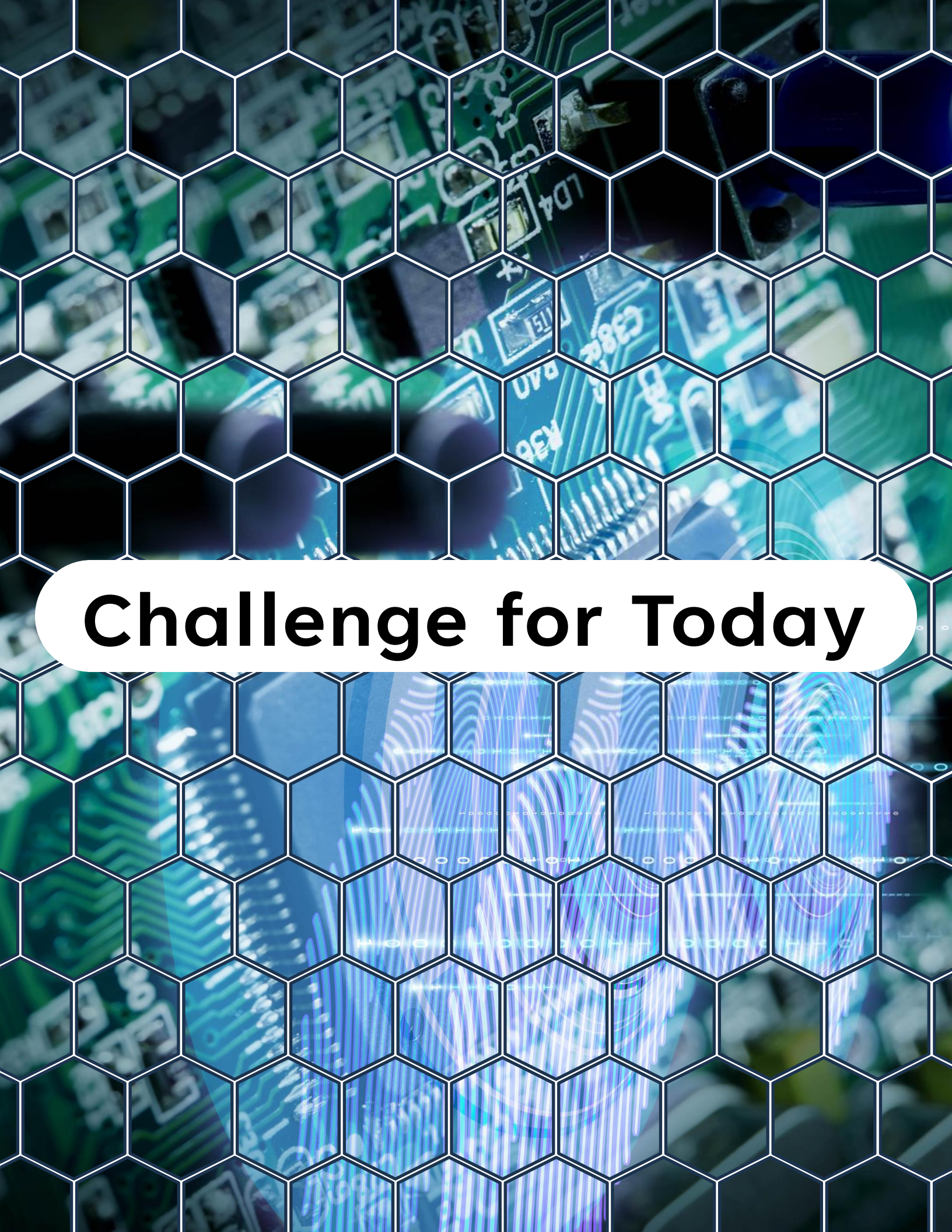
- ✓ Use **queues** for structured data (sensor readings).
- ✓ Use **semaphores** for event counting (new samples).
- ✓ Use **mutexes** for protecting shared resources (UART, I²C, SPI).
- ✓ Use **event groups** for multi-condition synchronization.
- ✓ Keep ISRs short — only signal via FromISR APIs, let tasks process the data.



Summary

Summary

- Real-world systems often combine **multiple FreeRTOS primitives**.
- Queues = data transfer, semaphores = event counting, mutexes = protection, event groups = synchronization.
- Proper use of inter-task communication avoids race conditions, corruption, and deadlocks.
- ESP32 applications benefit from carefully mixing these primitives to match design needs.



Challenge for Today

Challenge for Today

✂ Extend the project by:

1. Adding a **button ISR** that uses a **binary semaphore** to trigger an emergency log message.
2. Use a **task notification** to wake a “watchdog task” every 2 seconds from the tick hook.
3. Observe how all primitives (queues, semaphores, mutexes, event groups, task notifications) work **together in one system**.



"Thanks for watching!

*If you enjoyed this video, make sure to hit
the like button and subscribe to stay
updated with my latest content.*

*Don't forget to check out my other videos
for more tips and tutorials on Embedded
C, Python, hardware designs, etc. Keep
exploring, keep learning, and I'll see you in
the next video!"*

<https://www.linkedin.com/in/yamil-garcia>
<https://www.youtube.com/@LearningByTutorials>
<https://github.com/god233012yamil>